

COMMERCIAL HOSPITALITY SPECIFICATION

OZLODGE PLATFORM

Data-Sovereign Commercial Smart-Lock Infrastructure

Author:	Truong Viet Phan (Vince Phan)
Organization:	eBizco Australia Pty Ltd
Date:	August 2026
Status:	Technical Product Specification — Version 2.4

OZLODGE is the commercial hospitality tier of the Sovereign Edge smart-lock platform, engineered specifically for hotels, motels, short-stay accommodations, student housing, and mining accommodation. Grounded in the Sovereign Edge trust model, the platform resolves the post-purchase "hold-up" problem through a combination of open local-first runtime control, open-source compliance under GPLv3 copyleft licenses, and hardware credential containment. **This document serves as the formal architectural specification for Version 2.4.**

1. EXECUTIVE SUMMARY

1.1. Introduction & Core Philosophy

1.1.1. The Sovereign Edge Mission

Connected hardware products routinely place consumers and commercial buyers in a position that transaction cost economics calls "hold-up" (Klein et al., 1978; Williamson, 1979). Once a buyer has made a relationship-specific investment—such as installing locks and wiring mesh hubs—the vendor who controls the supporting cloud can unilaterally alter terms, degrade functionality, increase subscription rates, or withdraw support altogether, rendering the hardware useless. The smart-home and commercial access control industry's two dominant governance models each solve one dimension of this problem while sacrificing the other: white-label platforms offer low-cost market entry but force dependency on vendor-controlled cloud infrastructure; proprietary standards enable local control but gate market entry behind steep membership fees governed by incumbent firms.

1.1.2. Enterprise Scale-up Overview

OZLODGE addresses this fundamental gap in the smart-lock market. Designed specifically for commercial portfolios, it delivers the same sovereignty promise as the consumer tier—end-to-end encryption, local lock authority, and no plaintext credentials on any server—but at a commercial scale, with Property Management System (PMS) integration, automated guest check-in, and centralized front-desk control. Unlike white-label platforms (Tuya/TTLock) where a third-party vendor controls every lock globally and stores guest PINs in their cloud, OZLODGE puts control in the hands of the hotel operator. The hotel's own on-premises server is the ultimate authority, and the cloud operates strictly as a content-blind message relay. This is data sovereignty delivered as a commercial product.

1.2. The Sovereignty Promise

1.2.1. Architectural Safeguards

The platform makes strict operational guarantees, enforced by architecture rather than contract. No one can clone a lock because its eFuse private key is unique and unextractable in silicon. No one can see a guest's PIN because credentials are AES-GCM encrypted and never stored in the cloud. No one can forge a command because all control messages require cryptographic signatures. If the cloud vendor fails or alters terms, the hotel can self-host the open-source relay code without vendor cooperation. The locks operate 100% offline with local Bluetooth (BLE) and physical PIN keypad entry, removing any internet dependency.

1.2.2. Differentiated Licensing

To secure this open-open paradigm, OZLODGE utilizes a differentiated licensing strategy. The core protocol specification and client SDK are published under the permissive MIT License, encouraging adoption and integration. Conversely, the server codebase, bridge firmware, and mobile app are released under the strong copyleft GPLv3 License. GPLv3's anti-tivoization clause legally prevents the manufacturer from closing the platform or signing hardware in a way that blocks legitimate reflashing. Hardware schematics are governed by the CERN Open Hardware Licence Version 2 (CERN-OHL-S v2) to prevent proprietary closed hardware clones, and documentation is licensed under CC-BY-SA 4.0.

2. SYSTEM ARCHITECTURE

2.1. Network Topologies & Channels

2.1.1. Direct Local LAN Path

Staff tablets running the primary MAOI Property Management System (PMS) app and check-in kiosks communicate directly with the local server (ozlodgeserv) over the hotel's local Wi-Fi LAN. This connection is completely offline, ensuring that front-desk check-ins, room assignments, and physical key programming operate uninterrupted even if the property's internet connection fails entirely. All local LAN traffic is encrypted and requires initial BLE pairing during staff device enrollment.

2.1.2. Cloud-Relayed Path

For off-site or remote administrative operations, the MAOI PMS app communicates with the on-premises ozlodgeserv through the secure NEXUS cloud portal, utilizing ozlockserv as a secure STUN/MQTT conduit. Commands are sealed end-to-end from the tablet to the local server, meaning the cloud relay is structurally incapable of decrypting administrative payloads or forging commands.

2.1.3. Guest Interaction Path

Guests use the multipurpose BANOI app. For daily access, BANOI communicates directly with the door lock over Bluetooth (BLE) utilizing a locally-stored digital passport (X25519 member bond) and requiring no network connection. For remote operations, requests route from BANOI up to the routing-only cloud relay, down to the hotel's on-premises ozlodgeserv, and down-mesh to the target lock. This guest check-in path uses GPS geofencing to verify that the guest is physically within a 500-meter radius of the property before enabling check-in.

2.2. Component Descriptions & Licensing Splits

2.2.1. Smart Lock Hardware (ESP32-C6 N8)

Physical door locks are built on Espressif ESP32-C6 N8 microcontrollers operating in Full Thread Device (FTD, rx_on=1) mode for sub-second responsive mesh communication. PIN keypad entry and RFID cards are processed 100% offline against the lock's local database and stored public keys, ensuring immediate access even during catastrophic network outages.

2.2.2. Wi-Fi / Thread Bridge (ESP32-C6 N16)

Mains-powered ESP32-C6 N16 gateway devices acting as local Thread Border Routers and MQTT uplinks. Bridges route end-to-end encrypted payloads down to individual locks. Typical deployment density ranges from 8 to 16 locks per bridge, depending on physical floor layouts.

2.2.3. On-Premises ozlodgeserv

The physical local hardware brain on hotel premises (renamed from its draft designation 'OZKEYSERV'). Running locally on port :3200, it manages the physical lock network (via Thread/Wi-Fi bridges), holds room roster syncs, stores the local backup PMS database, and interfaces with third-party payment and SMS gateways locally. This server is the hotel's on-site system of record.

2.2.4. Open-Source ozlockserv Relay

The open-source (GPLv3) cloud server. It acts strictly as a content-blind message relay and a public-key directory. It stores only a simple mapping of device IDs to app IDs for message routing, and the public keys indexed by MAC addresses. It has no command decryption keys, no transaction logs, and no guest credentials, satisfying key provisions of the EU Cyber Resilience Act (CRA) at the architectural layer.

2.2.5. Proprietary NEXUS AI Layer

The commercial, closed-source artificial intelligence suite. It sits adjacent to the open-source relay and handles value-added, margin-rich operational logic such as AI-driven Interactive Voice Response (IVR) check-ins, intent analysis, and hospitality pattern learning. If a hotel exercises their self-hosting exit option by pointing to a private instance of ozlockserv, their locks continue to function flawlessly, but they lose access to the subscription-gated NEXUS AI features, preserving commercial SaaS margins.

2.2.6. MAOI PMS App

The core front-desk property management software running on staff tablets. MAOI serves as the ultimate authority for staff operations—allocating room assignments, managing shift rosters, issuing credentials, and reviewing audit trails. It has a local primary database which is backed up to the ozlodgeserv PMS database. A secondary variation of the MAOI app is deployed on public lobby Self-Check-In Kiosks.

2.2.7. BANOI App

A multipurpose guest-facing mobile application acting as a digital key container and managing check-ins and BLE unlocks. BANOI manages personal residential smart locks, performs commercial room bookings, and manages check-ins. In-stay guest BLE unlocks negotiate directly with locks via BLE.

2.3. System Architecture Diagram

2.3.1. Topology Visual Layout — Encrypted End-to-End Flow

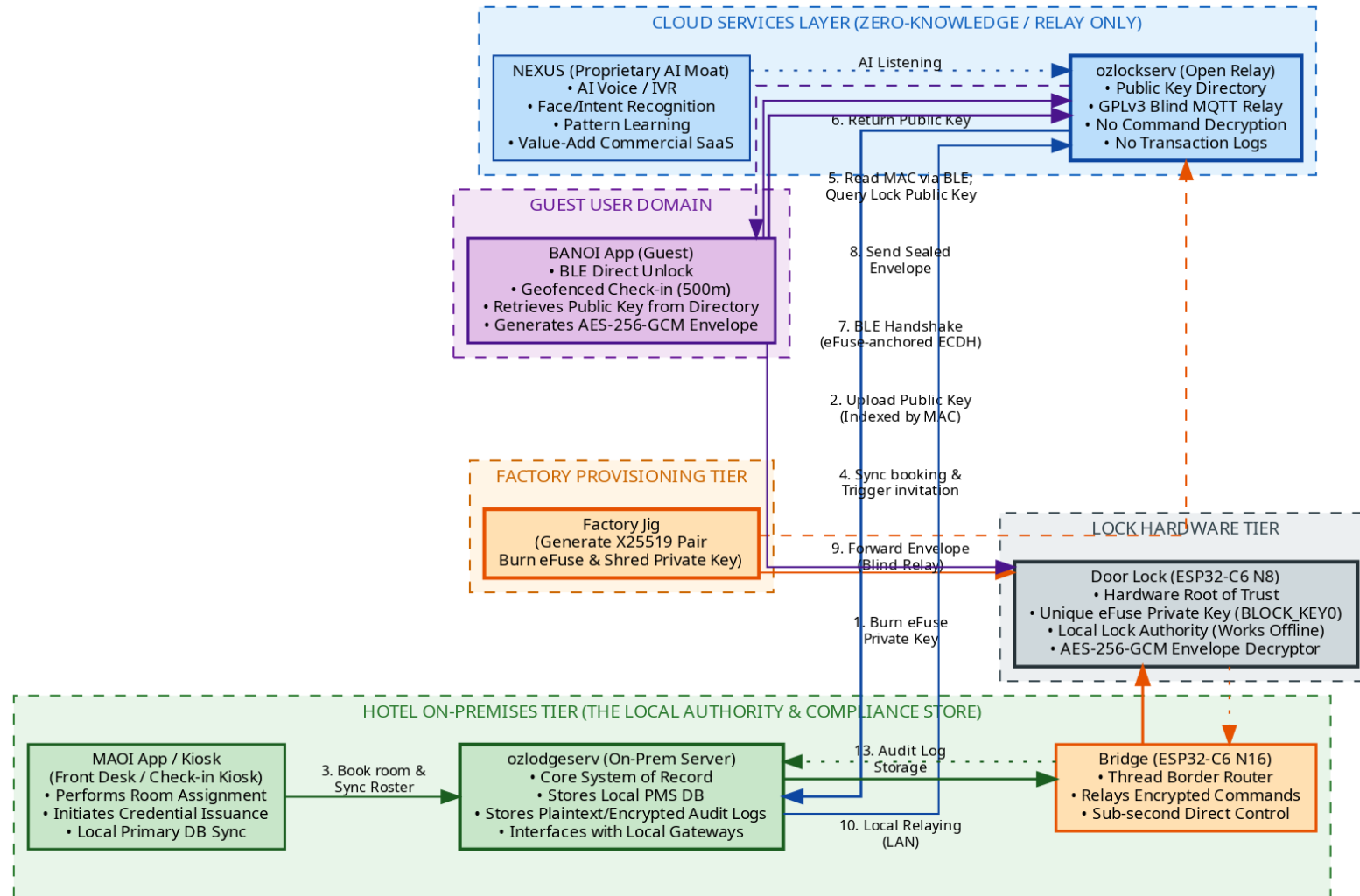


Figure 3: Detailed OZLODGE Hospitality Architecture showing factory provisioning, local LAN, routing-only cloud relay, and guest-application interfaces.

3. TRUST & SECURITY MODEL

3.1. Cryptographic Enforcements

3.1.1. AES-256-GCM Envelope Encryption

All control payloads (such as lock, unlock, or credential programming commands) are encrypted on the sender device (BANOI or MAOI) using AES-256-GCM before transport. The keys are derived locally via ECDH, ensuring the cloud relay cannot inspect or manipulate commands. Monotonic counters in the authenticated additional data (AAD) prevent any replay attacks, and commands are cryptographically bound to the target device ID via AAD. The server relays the hex envelope verbatim, maintaining structural content blindness.

3.1.2. X25519 eFuse Hardware Root of Trust

Every physical lock contains a unique, unextractable X25519 private key burned directly into the ESP32-C6 microcontroller's eFuse BLOCK_KEY0 with USER purpose (read/write protected by default). During manufacturing, each lock's corresponding public key is uploaded to the public key directory on ozlockserv. The private key never leaves the factory programming jig's ephemeral storage and is securely erased after burning using "shred -u". Cloned hardware is structurally locked out because it lacks the unique physical key in silicon required to compute the shared secret.

3.1.3. Local Microcontroller Lock Authority

The absolute authority to grant entry is localized on the lock's microcontroller. Credentials (PINs, RFID tokens, and member bonds) are stored in the lock's local flash. Expiry enforcement and digital passports are checked offline, meaning locks operate fully independently of a persistent network connection.

3.2. Data Sovereignty and Minimization

3.2.1. The Trust Model Fork

The platform maintains a strict zero-knowledge model on the cloud-side relay (ozlockserv), which has no visibility into commands and stores absolutely no transaction logs. However, the physical ozlodgeserv local server running inside the hotel's own physical LAN is a trusted, local authority. It legitimately consumes the publishLog() events sent from the locks via local Thread bridges to compile the PMS audit trail. Because the hotel physically owns and secures this local server on-site, compiling logs locally is not a contradiction of content-blindness; it is the ultimate expression of data sovereignty—keeping guest records out of the public cloud and within physical hotel custody.

3.2.2. Plaintext Log Transmission Gap

Currently, door opening events (publishLog()) are transmitted from the lock to the server in plaintext. On the residential tier (OZLOCK), the remediation is to stop publishing these log messages entirely since nothing in that architecture consumes them. However, on the commercial tier (OZLODGE), hotel operators legitimately require these logs. Therefore, the remediation for commercial properties is to secure these log topics via broker Access Control Lists (ACLs) and encrypted uplink channels, restricting access solely to the hotel's authorized on-premises ozlodgeserv.

4. KEY OPERATIONAL WORKFLOWS

4.1. Traditional Booking & Check-In

4.1.1. OTA Reservation Intake

A customer books a room on an Online Travel Agency (OTA) like booking.com. The OTA sends an email confirmation to the hotel, and the front-desk operator inputs the reservation details directly into the MAOI PMS app, which synchronizes it to the local primary database and backs it up on the local ozlodgeserv.

4.1.2. Lobby Kiosk Self-Check-In

If a guest arrives after-hours when the front desk is closed, they walk up to a lobby Self-Check-In Kiosk (Secondary MAOI App). The kiosk queries the local ozlodgeserv database over the LAN, verifies the registration, and generates their physical RFID keycard or programs their unique PIN directly into the lock. This is fully offline-compatible and requires no internet connection.

4.2. Mobile BANOI App Self-Check-In

4.2.1. Geofenced App Activation

If the hotel operator elects mobile check-in, the ozlodgeserv server registers the guest's booking and alerts NEXUS, which triggers an automated guest invitation via SMS or email containing a BANOI App download link. Upon account registration, the hotel booking is synchronized down to the guest's booking calendar page. When the guest arrives within a 500-meter radius of the property, the BANOI App displays a 'CheckIn' button via GPS geofencing.

4.2.2. Key Generation & eFuse Handshake

The guest taps check-in, sending an encrypted handshake from BANOI → NEXUS (acting as MQTT relay) → ozlodgeserv. BANOI reads the lock's MAC address via BLE, queries the public-key directory on ozlockserv to retrieve the lock's corresponding factory-uploaded public key, and executes the eFuse-anchored X25519 ECDH handshake to derive a pairing secret. From there, HKDF-SHA256 derives direction-specific App-to-Lock and Lock-to-App keys. The local server updates the lock with the digital key and validity duration in advance. At the door, the guest's BANOI app pairs directly with the lock via BLE and performs a secure, sub-second unlock completely offline.

5. COMPARISON & DIFFERENTIATION

5.1. Comparative Analysis Matrix

Feature / Metric	OZLODGE Hospitality Platform	Tuya / TTLock (White-Label)
Cloud Owner	Hotel / Host / Self-hosted on private VPS	Tuya (Third-party PRC corporate cloud)
Credential Storage	Encrypted at rest on local server; never in cloud	Plaintext PINs and RFID codes stored centrally
Server Visibility	Content-blind relay; routes sealed envelopes only	Server reads every command, PIN, and door event
Offline Operation	Works offline via BLE (ECDH) and lock MCU PINs	Requires active internet and cloud server validation
Data Sovereignty	Hotel owns and retains custody of audit logs	Third-party vendor retains control of usage data
Geopolitical Risk	None; offline LAN option & private server deployment	High; subject to PRC jurisdiction and export bans
PMS Integration	Deep local sync via /pms/rooms (upsert/reconcile)	Limited or complex proprietary API tethers

5.2. Moat & Strategic Licensing

By publishing core protocol specifications and client SDKs under the permissive MIT license, OZLODGE removes all ex-ante entry barriers and certification costs, encouraging competing hardware manufacturers to build compatible components. Conversely, publishing the server, bridge, and application code under the copyleft GPLv3 license establishes a strong 'Williamsonian hostage.' The anti-tivoization clause of GPLv3 prevents the vendor from later locking down the platform via secure boot or closing the source code. This license architecture guarantees that hotels are never trapped in a monopolistic relationship, capping support margins at whatever the competitive technical support market bears. Hardware schematics are governed by CERN-OHL-S v2 to prevent closed clones.

6. SYSTEM IMPLEMENTATION STATUS

6.1. Revision History & Retrospective

The August 2026 revisions (v2.1 to v2.4) incorporated critical corrections from security and hardware audits. First, the dead-name 'OZKEYSERV' was globally replaced with the clean, domain-specific 'ozlodgeserv.' Second, the 'Sleepy End Device' claim was corrected to reflect that the lock operates as a 'Full Thread Device' (rx_on=1) to guarantee a sub-second mesh command relay. Third, the physical eFuse root of trust was integrated, correcting the key-purpose configuration from ECDSA_KEY to USER so the hardware ECDH peripheral can legally access the block and complete the key-exchange. Fourth, the Trust Fork was formally documented to clarify how cloud blindness and local audit logging co-exist.

6.2. Status Registers

6.2.1. Shared Platform Claims (C1–C18)

Claim	Description	Status	Evidence / Reference
C1	Commands are AES-256-GCM sealed end-to-end	VERIFIED	ozkey-06 envelope validation
C2	Server relays envelope_hex verbatim, blind relay	VERIFIED	buildCredentialFrame() deleted from relay
C3	Monotonic counter prevents replay attacks	VERIFIED	Counter_floor per bond; written to U0 block
C4	Credentials never stored in plaintext on server	VERIFIED	Relay database stores metadata only
C5	Lock works offline via BLE and PIN keypad	VERIFIED	Credentials verified locally on MCU
C6	Lock-to-app communication is sealed	VERIFIED	ozkey-17 U1 secure channel
C7	Server stores no record of door events	CORRECTED	publishLog() dropped on OZLOCK tier via guard
C8	Sleepy End Device power optimization	INCORRECT	Firmware runs FTD (rx_on=1) for ~300ms latency
C9	Multi-year battery life in FTD mode	UNMEASURED	Requires field measurement under mesh traffic
C11	Key burned into BLOCK_KEY0 with USER purpose	PENDING	Requires factory jig update to burn USER
C12	NEXUS stores public keys, never private keys	VERIFIED	NEXUS schema has public_key_hex column only
C18	Forward secrecy is not claimed (residential trade-off)	ACKNOWLEDGED	Explicit limitation documented in Rev 3.2

6.2.2. Hospitality Tier Claims (L1-L10)

Claim	Description	Status	Evidence / Reference
L1	Hotel server cannot read credentials in transit	VERIFIED	Inherits C1/C2 zero-knowledge envelopes
L2	PIN stored on lock MCU, works 100% offline	VERIFIED	DP 21/22 path, hardware-verified
L3	Digital passport is X25519 member bond	VERIFIED	M3 member ceremony verified on-device
L4	Access revocation is executed lock-side	VERIFIED	DP 101 bond_revoke removes local keys
L5	Remote unlock is sub-second via bridge	VERIFIED	Measured ~240-330 ms lock-side relay
L6	Lock sleeps with configurable interval	INCORRECT	Firmware is rx_on=1; 60s is heartbeat rate
L7	Multi-year battery at hospitality duty cycle	UNMEASURED	Inherits C9 unmeasured status in FTD mode
L8	Full audit trail stored on hotel's local server	ACHIEVABLE	Trust Fork explicitly documented in v2.4
L9	After-hours QR self-check-in flow	DESIGN ONLY	Phase 5 roadmap, implementation pending
L10	PMS room roster sync /pms/rooms endpoint	BUILT	Supports upsert and reconcile modes successfully